Foundational concepts of machine learning

Lecture Notes

Giorgio Gambosi

Master's Degree in Computer Science

University of Rome "Tor Vergata"

A.Y. 2026–2027

Contents

1 Introductory concepts **1**
- 1.1 Three paradigms of machine learning . . . 1
- 1.2 Data representation . . . 2
- 1.3 The learning process and generalization . . . 2
- 1.4 Formal definition of a supervised learning task . . . 2
- 1.5 Relations between items and their targets . . . 3
- 1.6 Functional predictors . . . 3
- 1.7 Probabilistic predictors . . . 4

2 Measuring the quality of a predictor: the loss function **5**
- 2.1 Loss function definition, per-element risk, and expected risk . . . 6
- 2.2 The empirical risk as an estimator of the expected risk . . . 6
- 2.3 Minimization of the empirical risk and its issues . . . 9
- 2.4 Bayes predictors and Bayes risk . . . 13

3 The bias-variance decomposition **14**
- 3.1 The general decomposition . . . 14
- 3.2 Interpretability . . . 14

1 Introductory concepts

Machine learning rests on a simple observation: data is rarely random. Whether you analyse images, text, sensor measurements, or human behaviour, the data you collect contains patterns reflecting the underlying process that generated it. The goal of machine learning is to infer such patterns from data without explicitly programming the rules.

From a collection of observations we construct **models**: mathematical abstractions capturing the relationships present in the data, used for prediction and decision-making on new, unseen data.

1.1 Three paradigms of machine learning

1.1.1 Supervised Learning

In supervised learning we work with paired data: observations (as feature sets) and corresponding target values. The goal is to learn a mapping from inputs to targets and predict unseen cases. For instance, given houses described by square footage, number of bedrooms, and location, each with a known selling price, we learn a function mapping features to prices and use it to estimate the price of a new house.

Supervised learning splits into two categories depending on the nature of the target:

Regression The target is a real number (continuous): prices, temperatures, stock values, or other continuous quantities.

Classification The target is a discrete category: spam detection, handwritten digit recognition, diagnosis, sentiment analysis.

In both cases, we define a model: either a function $y(\mathbf{x})$ outputting a prediction directly, or a probability distribution $p(t|\mathbf{x})$ quantifying uncertainty about the target value. Training data are then used to adjust the model's parameters.

1.1.2 Unsupervised Learning

Unsupervised learning works without explicit targets, looking for structure in unlabelled data: clustering groups similar items; density estimation models the distribution generating the data; dimensionality reduction projects data onto lower-dimensional representations preserving key information (**Principal Component Analysis (PCA)**, for instance, retains the top k orthogonal directions of greatest variance).

1.1.3 Reinforcement Learning

In reinforcement learning, rather than working with a fixed dataset, an agent learns through interaction with an environment, repeatedly choosing actions and adapting its behaviour to maximize cumulative reward. At each step the agent observes the current state and selects an action; the environment returns a reward and a new state. These signals guide the agent toward an optimal policy $\pi : S \to A$ maximizing expected cumulative reward:

$$\pi^* = \underset{\pi}{\operatorname{argmax}}\, \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \right] \tag{1}$$

where $\gamma \in [0, 1]$ is a discount factor balancing immediate versus future rewards, and r_t is the reward at step t. The central tension of this framework is the exploration-exploitation tradeoff, and since it differs substantially from the other two paradigms these notes set it aside.

1.2 Data representation

Every machine learning method starts with a data representation choice. Real-world entities (images, documents, customers, transactions, biological samples) must be converted into feature vectors. A feature vector $\mathbf{x} = (x_1, x_2, \ldots, x_d)$ is a collection of d numerical measurements describing an object, and its choice affects both prediction quality and computational efficiency. In image classification, for instance, raw pixel intensities yield millions of noisy features, while edge orientations or textures may be more informative. This is why feature engineering (combining, selecting or learning features) is often as important as the choice of learning algorithm.

1.3 The learning process and generalization

Across all paradigms we start with a training set $\mathcal{T} = (\mathbf{x}, \mathbf{t})$ containing n examples. In supervised learning, each example pairs an input feature vector $\mathbf{x}_i$ with its target value t_i; in unsupervised learning, we have only the feature vectors $\mathbf{x} = \{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$.

The key challenge is not fitting the training data but generalizing to unseen data. A model memorising the training data fails on new examples; an overly simple one misses important patterns. The capacity of a model determines its flexibility, the bias-variance tradeoff reflects the tension between underfitting and overfitting, and regularisation penalises complexity to improve generalization. These notions are made precise in the rest of these notes.

1.4 Formal definition of a supervised learning task

A supervised learning task involves two key sets: the domain set $\mathcal{X}$ and the label set $\mathcal{Y}$. The domain set collects the objects to be labelled; each object is modelled as an array of features $\mathbf{x} = (x_1, x_2, \ldots, x_d)$, where d is the dimensionality (more complex structures, such as sequences or graphs, are also possible). The label set contains all possible output values: if $\mathcal{Y}$ is continuous we face a regression task, and when $\mathcal{Y}$ is discrete a classification task — binary when $|\mathcal{Y}| = 2$ and multi-class when $|\mathcal{Y}| > 2$.

The learning algorithm $\mathcal{A}$ receives a training set $\mathcal{T} = \{(\mathbf{x}_1, t_1), \ldots, (\mathbf{x}_n, t_n)\}$, organised into a feature matrix $\mathbf{X}$ (objects as rows) and a target vector $\mathbf{t}$:

$$\mathbf{X} = \begin{pmatrix} - & \mathbf{x}_1 & - \\ & \vdots & \\ - & \mathbf{x}_n & - \end{pmatrix}, \quad \mathbf{t} = \begin{pmatrix} t_1 \\ \vdots \\ t_n \end{pmatrix}. \tag{2}$$

The aim is to predict labels for unseen elements $\overline{\mathbf{x}} \in \mathcal{X}$, either by directly using the training set at prediction time or, more generally, by deriving from it a predictor used for the prediction.

1.5 Relations between items and their targets

Both the objects we observe and their labels come from some underlying stochastic process. The **input distribution** p_M describes the sampling likelihood of objects, while the simplest assumption for the target is a deterministic **ground truth function** $f : \mathcal{X} \to \mathcal{Y}$ such that $t = f(\mathbf{x})$ exactly. More realistically, target values may depend on unobserved variables, or observations may be noisy: the same object may then legitimately receive different labels. This is captured by a **conditional label distribution** $p_C(\cdot|\mathbf{x})$, which, together with p_M, induces the joint distribution $p_J(\mathbf{x}, t) = p_M(\mathbf{x})\, p_C(t|\mathbf{x})$ over object-label pairs. The functional setting is the special case in which $p_C(\cdot|\mathbf{x})$ places all its mass on $f(\mathbf{x})$.

1.6 Functional predictors

In its most natural form, a **functional predictor** is a map $h : \mathcal{X} \to \mathcal{Y}$ that assigns a prediction $\hat{y}$ to each input $\mathbf{x}$:

$$\mathbf{x} \xrightarrow{h} \hat{y} \in \mathcal{Y}. \tag{3}$$

For reasons that will become clear later, however, it is usually defined as a map $h : \mathcal{X} \to \mathbb{R}$ that assigns a real-valued **score** $y = h(\mathbf{x})$ to each input. The prediction $\hat{y} \in \mathcal{Y}$ is then obtained from the score through a task-dependent **decision rule** $d : \mathbb{R} \to \mathcal{Y}$:

$$\mathbf{x} \xrightarrow{h} y \xrightarrow{d} \hat{y} \in \mathcal{Y}. \tag{4}$$

A prediction function can be constructed according to three different strategies, schematized in Figures 1–3 with a common color code: training data in green, algorithms in blue, learned predictors in red, inputs and outputs in grey.

1.6.1 Direct prediction

In the simplest approach there is no learning phase. The training examples are stored, and a predefined algorithm computes the score $y = h(\mathbf{x}, \mathcal{T})$ at prediction time, using both the new item and the whole training set (Figure 1).

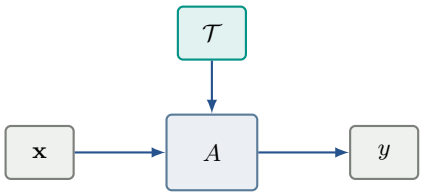


Figure 1: Direct prediction: the algorithm A receives both the new input $\mathbf{x}$ and the whole training set $\mathcal{T}$, and returns the score y. There is no separate learning phase.

A classic example is the **k-nearest neighbours** (k-NN) algorithm: a new item $\mathbf{x}$ is assigned the majority class among the k training items closest to it, according to a given distance. The approach is simple, makes no assumption on the form of the predictor, and adapts locally to the data, at the cost of processing the training set at every prediction.

1.6.2 Model-based learning

The second and most common approach fixes in advance a **hypothesis class** $\mathcal{H}$, a family of functions from $\mathcal{X}$ to $\mathcal{Y}$. A learning algorithm $\mathcal{A}$ uses the training set to select the function $h_{\mathcal{T}} \in \mathcal{H}$ that best fits the examples; afterwards, the training set is no longer needed, and prediction simply computes $h_{\mathcal{T}}(\mathbf{x})$ (Figure 2).

Often $\mathcal{H}$ consists of **parametric functions**, which share the same structure and differ only in the values of a parameter vector $\boldsymbol{\theta} = (\theta_1, \ldots, \theta_m)$. We then write $\mathcal{H}_{\boldsymbol{\theta}}$, and learning reduces to finding the parameter values that best fit the training data. In linear regression, for instance, the prediction is a weighted sum of the features,

$$y = \sum_{i=1}^{d} w_i x_i + w_0 = \mathbf{w}^T \overline{\mathbf{x}}, \tag{5}$$

where $\mathbf{w} = (w_0, w_1, \ldots, w_d)^T$ is the parameter vector and $\overline{\mathbf{x}} = (1, x_1, \ldots, x_d)^T$ is the input extended with a constant component, which accounts for the bias term w_0. Learning finds the weights minimizing the prediction error on the training set.

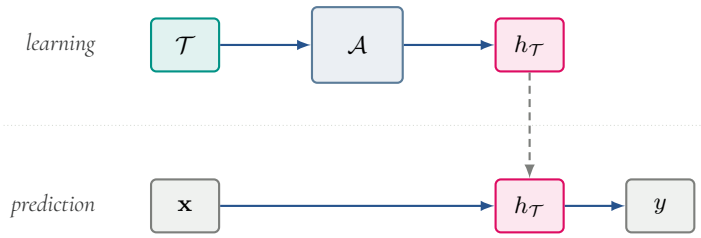


Figure 2: Model-based learning. In the learning phase (top), the algorithm $\mathcal{A}$ processes the training set $\mathcal{T}$ and returns a predictor $h_{\mathcal{T}} \in \mathcal{H}$. In the prediction phase (bottom), $h_{\mathcal{T}}$ maps each new input $\mathbf{x}$ to a score y, without accessing the training set.

1.6.3 Ensemble learning

The third approach builds several predictors and combines their predictions. Such an **ensemble** often outperforms each of its members, since different predictors make different errors, which partly compensate. In the learning phase, s predictors $h_{\mathcal{T}}^{(1)}, \ldots, h_{\mathcal{T}}^{(s)}$ are derived from the training set, and each is assigned a weight $w^{(i)}$ reflecting its estimated quality. In the prediction phase, the scores $y^{(i)} = h_{\mathcal{T}}^{(i)}(\mathbf{x})$ are computed and combined according to the weights (Figure 3).

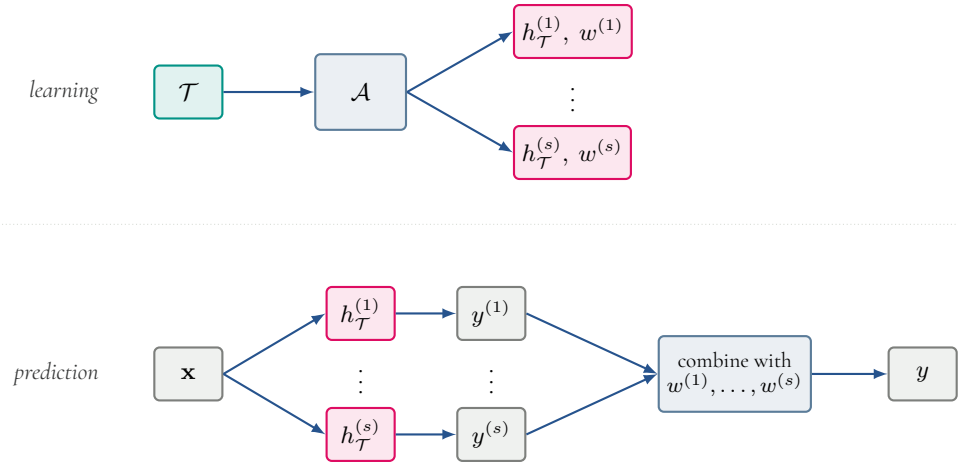


Figure 3: Ensemble learning. In the learning phase (top), the training set is used to derive s predictors $h_{\mathcal{T}}^{(1)}, \ldots, h_{\mathcal{T}}^{(s)}$, with weights $w^{(1)}, \ldots, w^{(s)}$. In the prediction phase (bottom), each predictor computes its own score $y^{(i)}$ on the new input $\mathbf{x}$, and the scores are combined according to the weights into the final score y.

In regression, the combination is a weighted average,

$$h(\mathbf{x}) = \sum_{i=1}^{s} w^{(i)} h_{\mathcal{T}}^{(i)}(\mathbf{x}), \tag{6}$$

with $w^{(i)} \geq 0$ and $\sum_{i=1}^{s} w^{(i)} = 1$. In classification, it is a **weighted vote**, which returns the class receiving the largest total weight:

$$h(\mathbf{x}) = \operatorname*{argmax}_{y \in \mathcal{Y}} \sum_{i=1}^{s} w^{(i)} \mathbb{1}[h_{\mathcal{T}}^{(i)}(\mathbf{x}) = y]. \tag{7}$$

In summary, direct methods are flexible and adapt locally to the data, model-based methods are efficient at prediction time and often interpretable, and ensemble methods are robust and often more accurate.

1.7 Probabilistic predictors

A **probabilistic predictor** is a map

$$h : \mathcal{X} \to \mathcal{D}(\mathcal{Y}), \tag{8}$$

where $\mathcal{D}(\mathcal{Y})$ denotes a set of probability distributions over $\mathcal{Y}$: for each input $\mathbf{x}$, the predictor returns a whole distribution $h(\mathbf{x}) = p_P(\cdot|\mathbf{x})$ on $\mathcal{Y}$, instead of a single value. A functional predictor is the special case in which all the probability mass is concentrated on a single value. As in the functional case, a prediction is obtained by applying a **decision rule**, here a map $d : \mathcal{D}(\mathcal{Y}) \to \mathcal{Y}$ applied to the distribution p_P:

$$\mathbf{x} \xrightarrow{h} p_P \xrightarrow{d} \hat{y} \in \mathcal{Y}. \tag{9}$$

The natural decision rule depends on the task.

Regression. For $\mathcal{Y} = \mathbb{R}$, the natural decision rule is the **posterior mean**,

$$\hat{y} = d(p_P) = \mathop{\mathbb{E}}_{t \sim p_P(\cdot|\mathbf{x})}[t]\,. \tag{10}$$

Classification. For $\mathcal{Y} = \{-1, 1\}$ in the binary case, and $\mathcal{Y} = \{y_1, \ldots, y_K\}$ in the multiclass case, the natural decision rule is the **posterior mode**, the most probable class,

$$\hat{y} = d(p_P) = \mathop{\text{argmax}}_{t \in \mathcal{Y}} p_P(t|\mathbf{x}). \tag{11}$$

The distribution p_P can be returned explicitly, as the values $p_P(y_k)$ when $\mathcal{Y}$ is discrete; through the parameters of a parametric family, such as the mean and variance of a Gaussian; or as a procedure computing $p_P(t)$ for any t. The three strategies described above apply to probabilistic predictors as well.

1.7.1 Direct prediction

An algorithm receives $\mathbf{x}$, a target value t, and the whole training set $\mathcal{T}$, and returns $p_P(t|\mathbf{x})$ directly, without a separate learning phase (Figure 4).

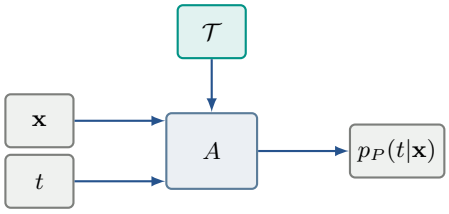


Figure 4: Direct probabilistic prediction: the algorithm A receives the new input $\mathbf{x}$, a target value t, and the whole training set $\mathcal{T}$, and returns the probability $p_P(t|\mathbf{x})$, without a separate learning phase.

An example is classification through density estimation methods such as **Parzen windows**, which estimate from the training set the class-conditional densities $p(\mathbf{x}|y_k)$; given the prior probabilities $p(y_k)$ of the classes, $p_P(y_k|\mathbf{x})$ then follows by Bayes' rule.

1.7.2 Model-based learning

Here the role of the hypothesis class $\mathcal{H}$ is played by a **parametric family** $\mathcal{D}$ of conditional distributions $p(\cdot|\cdot;\boldsymbol{\theta})$. Given a training set $\mathcal{T}$, the learning algorithm $\mathcal{A}$ selects the distribution $p^*(\cdot|\cdot;\mathcal{T}) \in \mathcal{D}$ that best fits the training examples, according to a criterion discussed later. At prediction time, the distribution returned for an item $\mathbf{x}$ is $p_P = p^*(\cdot|\mathbf{x};\mathcal{T})$ (Figure 5).

1.7.3 Ensemble learning

The **fully Bayesian** approach, introduced later, is an instance of ensemble learning: instead of selecting a single distribution, it derives $p_P(\cdot|\mathbf{x})$ by averaging the predictions of all the distributions in a parametric family, each weighted by its posterior probability given the training set.

2 Measuring the quality of a predictor: the loss function

Learning means selecting a predictor within some class, which requires measuring how good a prediction is. This is done through the **risk**, the error a prediction incurs on a randomly chosen item, built on a **loss function** $L(h(\mathbf{x}), t)$ measuring how far the prediction for $\mathbf{x}$ is from its true output t. When $h(\mathbf{x})$ is a distribution, the comparison function is called a **scoring rule**, though "loss function" is often used loosely for both.

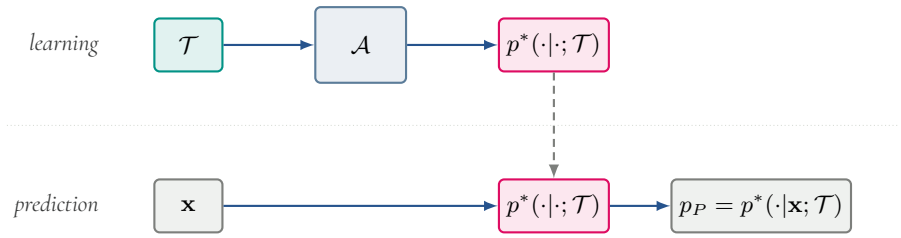


Figure 5: Model-based probabilistic learning. In the learning phase (top), the algorithm $\mathcal{A}$ processes the training set $\mathcal{T}$ and selects a conditional distribution $p^*(\cdot|\cdot;\mathcal{T})$ in the family $\mathcal{D}$. In the prediction phase (bottom), the distribution $p_P = p^*(\cdot|\mathbf{x};\mathcal{T})$ is returned for each new input $\mathbf{x}$, without accessing the training set.

2.1 Loss function definition, per-element risk, and expected risk

The loss function L compares a prediction y — a score in $\mathbb{R}$, a target value in $\mathcal{Y}$, or a distribution on $\mathcal{Y}$ — with the target value $t \in \mathcal{Y}$, returning the cost of predicting y when the target is t[1]. The per-element risk $\mathcal{R} : \mathcal{X} \mapsto \mathbb{R}$ of a predictor h, on a single item $\mathbf{x}$ is the error we expect h to incur when applied on input $\mathbf{x}$: under a functional relation it is simply

$$\mathcal{R}(\mathbf{x}; h) = L(h(\mathbf{x}), f(\mathbf{x})) \tag{12}$$

under a probabilistic relation it is the expectation of $L(h(\mathbf{x}), t)$ over $t \sim p_C(\cdot|\mathbf{x})$,

$$\mathcal{R}(\mathbf{x}; h) = \mathop{\mathbb{E}}_{t \sim p_C(\cdot|\mathbf{x})} \left[L(h(\mathbf{x}), t) \right]. \tag{13}$$

The expected risk for the predictor h is the expectation of the per element risk $\mathcal{R}(\mathbf{x})$ over all $\mathbf{x} \sim p_M$

$$\mathcal{R}(h) = \mathop{\mathbb{E}}_{\mathbf{x} \sim p_M} \left[\mathcal{R}(\mathbf{x}; h) \right] = \mathop{\mathbb{E}}_{\mathbf{x} \sim p_M} \left[\mathop{\mathbb{E}}_{t \sim p_C(\cdot|\mathbf{x})} \left[L(h(\mathbf{x}), t) \right] \right] = \mathop{\mathbb{E}}_{(\mathbf{x},t) \sim p_J} \left[L(h(\mathbf{x}), t) \right] \tag{14}$$

this is the average error of the predictor on a randomly chosen item, our main theoretical indicator of model quality. Computing it, however, requires the knowledge of $p_J(\mathbf{x}, t) = p_C(t|\mathbf{x}) p_M(\mathbf{x})$, which is unavailable by assumption; a related indicator, the empirical risk, is used in practice.

Four cases arise, depending on the relation between item and target (functional vs. probabilistic) and on the type of predictor (functional vs. probabilistic), governed by a single principle: the **type of predictor** determines the structure of the loss (a value or score y compared with t by a loss function, or a distribution p_P compared with t by a scoring rule), while the **relation model** determines the per-element risk (the loss evaluated at $t = f(\mathbf{x})$, or an expectation over $p_C(\cdot|\mathbf{x})$). Table 1 summarizes the four cases.

2.2 The empirical risk as an estimator of the expected risk

The expected risk depends on L, on p_M and on p_C (or f). L is ours to choose and p_M can usually be assumed known, but f and p_C are unknown by assumption, otherwise the task would be trivial. What we do have is the empirical risk: the average loss over the training set, viewed as a sample drawn from p_J,

$$\overline{\mathcal{R}}_{\mathcal{T}}(h) = \frac{1}{|\mathcal{T}|} \sum_{(\mathbf{x},t) \in \mathcal{T}} L(h(\mathbf{x}), t). \tag{15}$$

This is a sensible substitute, since the expected risk factors as

$$\mathop{\mathbb{E}}_{(\mathbf{x},t) \sim p_J} \left[L\big(h(\mathbf{x}), t\big) \right] = \mathop{\mathbb{E}}_{\mathbf{x} \sim p_M} \left[\mathop{\mathbb{E}}_{t \sim p_C(\cdot|\mathbf{x})} \left[L\big(h(\mathbf{x}), t\big) \right] \right], \tag{16}$$

and the inner expectation reduces to $L(h(\mathbf{x}), z)$ whenever $p_C(\cdot|\mathbf{x})$ places all its mass on z. Hence the empirical risk is the sample mean used as an estimator of the expectation defining the risk, with the same definition in all four cases (Table 2).

The overall framework is drawn in Figure 6 for the case of an assumed functional relation $t = f(\mathbf{x})$ between item and target and in Figure 7 for the more general case of a probabilistic relation $t \sim p_C(\cdot|\mathbf{x})$.

[1] As we shall see, in classification, where $\mathcal{Y}$ is discrete, a loss function comparing pairs of discrete values lacks properties necessary for learning. Thus, functional predictors are defined as predicting real scores — that the decision rule d maps to values in $\mathcal{Y}$ — which are then referred to in the surrogate loss function.

Setting	Loss	Per-element risk $\mathcal{R}(\mathbf{x}; h)$	Expected risk $\mathcal{R}(h)$
FF case functional predictor functional relation	$L\big(y,\, t\big)$	$L\big(h(\mathbf{x}),\, f(\mathbf{x})\big)$	$\mathop{\mathbb{E}}_{\mathbf{x}\sim p_M}\big[L\big(h(\mathbf{x}), f(\mathbf{x})\big)\big]$
FP case functional predictor probabilistic relation	$L\big(y,\, t\big)$	$\mathop{\mathbb{E}}_{t\sim p_C(\cdot\vert\mathbf{x})}\big[L\big(h(\mathbf{x}), t\big)\big]$	$\mathop{\mathbb{E}}_{(\mathbf{x},t)\sim p_J}\big[L\big(h(\mathbf{x}), t\big)\big]$
PF case probabilistic predictor functional relation	$S\big(p_P, t\big)$	$S\big(h(\mathbf{x}), f(\mathbf{x})\big)$	$\mathop{\mathbb{E}}_{\mathbf{x}\sim p_M}\big[S\big(h(\mathbf{x}), f(\mathbf{x})\big)\big]$
PP case probabilistic predictor probabilistic relation	$S\big(p_P, t\big)$	$\mathop{\mathbb{E}}_{t\sim p_C(\cdot\vert\mathbf{x})}\big[S\big(h(\mathbf{x}), t\big)\big]$	$\mathop{\mathbb{E}}_{(\mathbf{x},t)\sim p_J}\big[S\big(h(\mathbf{x}), t\big)\big]$

Table 1: The four combinations of relation model and predictor type. By definition, given a predictor h, $h(\mathbf{x}) = y$ for FF and FP, while $h(\mathbf{x}) = p_P$ for PF and PP. Moving across relation types changes only the per-element risk; moving across predictor types changes the loss from a scalar comparison L to a scoring rule S. The expected risk is always the expectation of the per-element risk with respect to p_M (recall that $p_J(\mathbf{x}, t) = p_M(\mathbf{x})p_C(t|\mathbf{x})$).

Setting	Loss	Prediction	Empirical risk $\overline{\mathcal{R}}_{\mathcal{T}}(h)$
FF case functional predictor functional relation	$L\big(y,\, t\big)$	$y = h(\mathbf{x})$	$\dfrac{1}{\vert\mathcal{T}\vert}\displaystyle\sum_{(\mathbf{x},t)\in\mathcal{T}} L(h(\mathbf{x}), t)$
FP case functional predictor probabilistic relation	$L\big(y,\, t\big)$	$y = h(\mathbf{x})$	$\dfrac{1}{\vert\mathcal{T}\vert}\displaystyle\sum_{(\mathbf{x},t)\in\mathcal{T}} L(h(\mathbf{x}), t)$
PF case probabilistic predictor functional relation	$S\big(p_P, t\big)$	$p_P = h(\mathbf{x})$	$\dfrac{1}{\vert\mathcal{T}\vert}\displaystyle\sum_{(\mathbf{x},t)\in\mathcal{T}} S\big(p_P, t_i\big)$
PP case probabilistic predictor probabilistic relation	$S\big(p_P, t\big)$	$p_P = h(\mathbf{x})$	$\dfrac{1}{\vert\mathcal{T}\vert}\displaystyle\sum_{(\mathbf{x},t)\in\mathcal{T}} S\big(p_P, t_i\big)$

Table 2: The empirical risk in all four combinations of relation model and predictor type. As can be seen, its structure is the same in all of them: comparing the prediction returned by h with the observed value of the target.

2.2.1 General properties of the sample mean for i.i.d. variables

We first recall general properties of the sample mean of i.i.d. random variables, which apply to the empirical risk as a particular sample mean.

Unbiasedness. Let Z be a random variable with expectation $\mathop{\mathbb{E}}_{Z\sim p}[Z] = \mu$, and let $Z_1, \ldots, Z_n$ be n independent draws from $p(\cdot)$. The average

$$\overline{Z} = \frac{1}{n}\sum_{i=1}^{n} Z_i \tag{17}$$

is an unbiased estimator of μ. That is,

$$\mathop{\mathbb{E}}_{(Z_1,\ldots,Z_n)\sim p^n}\big[\overline{Z}\big] = \mu. \tag{18}$$

Thus, the expectation of the sample average equals the true mean μ, which is precisely what it means for an

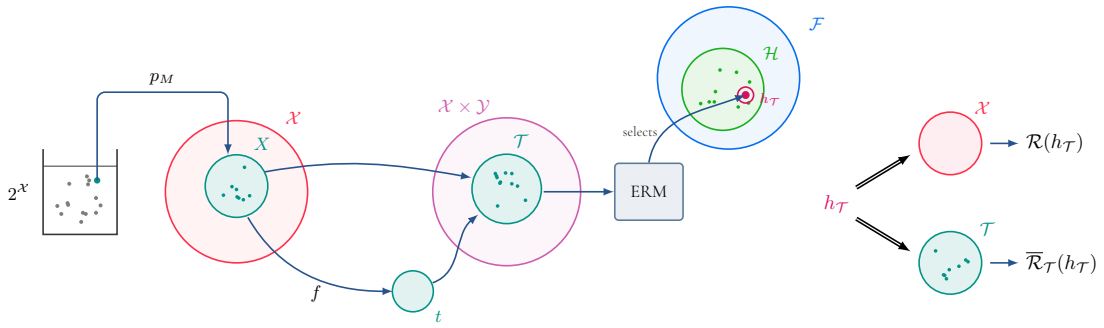


Figure 6: Learning with a functional relation. A set of items X is drawn from $\mathcal{X}$ according to p_M, and labeled by f, yielding the training set $\mathcal{T} \subset \mathcal{X} \times \mathcal{Y}$. ERM selects $h_\mathcal{T}$ within the hypothesis class $\mathcal{H} \subset \mathcal{F}$, the set of all (measurable) functions; its quality is measured by the risk $\mathcal{R}(h_\mathcal{T})$, over the whole $\mathcal{X}$, while ERM only has access to the empirical risk $\overline{\mathcal{R}}_\mathcal{T}(h_\mathcal{T})$, over $\mathcal{T}$.

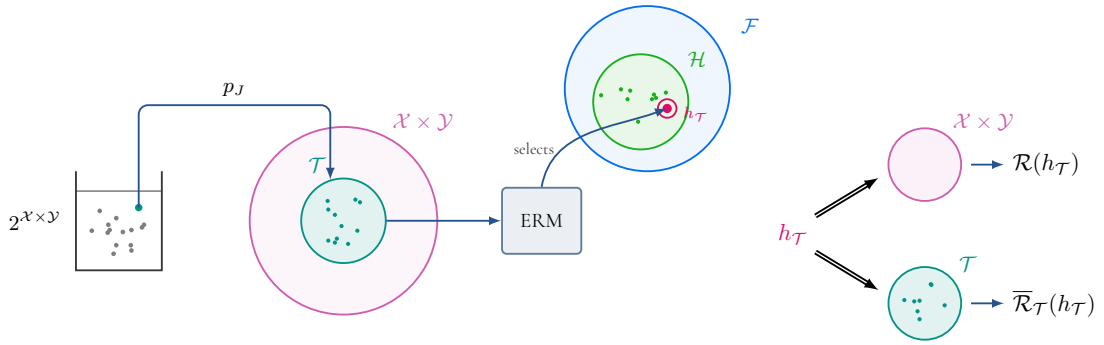


Figure 7: Learning with a probabilistic relation. The training set $\mathcal{T}$ is drawn directly from $\mathcal{X} \times \mathcal{Y}$ according to the joint distribution p_J. ERM selects $h_\mathcal{T}$ within the hypothesis class $\mathcal{H} \subset \mathcal{F}$, the set of all (measurable) functions; its risk $\mathcal{R}(h_\mathcal{T})$ is measured over $\mathcal{X} \times \mathcal{Y}$, its empirical risk $\overline{\mathcal{R}}_\mathcal{T}(h_\mathcal{T})$ over $\mathcal{T}$.

estimator to be unbiased. Note that independence is not required for this result, provided that each Z_i has expectation μ.

The same holds when $Z = g(X)$ for some fixed function g of another random variable X: if $X_1, \ldots, X_n$ are independent draws from the distribution of X, then averaging $g(X_1), \ldots, g(X_n)$ yields an unbiased estimator of $\mathbb{E}_X[g(X)]$, provided that the expectation exists.

Consistency. Unbiasedness says nothing about how close $\overline{Z}$ is to μ for a particular sample. What happens as the sample grows is described by consistency: a sequence of estimators $\hat{\mu}_n$ converges in probability to μ, written $\hat{\mu}_n \xrightarrow{p} \mu$, if for every $\varepsilon > 0$,

$$\lim_{n\to\infty} P\big(|\hat{\mu}_n - \mu| > \varepsilon\big) = 0. \tag{19}$$

By the Weak Law of Large Numbers, the average of n i.i.d. random variables with finite mean converges in probability to that mean. As a consequence,

$$\lim_{n\to\infty} P\big(|\frac{1}{n}\sum_{i=1}^{n} Z_i - \mu| > \varepsilon\big) = 0. \tag{20}$$

Concentration around the mean. Consistency says nothing about how quickly $\overline{Z}$ settles around μ; here the variance enters through the notion of concentration. The elementary tool quantifying this is Chebyshev's inequality.

Chebyshev's inequality and the variance of a sample mean

For *any* random variable X with finite mean μ and finite variance σ^2, and any $\varepsilon > 0$, Chebyshev's

inequality states

$$P\big(|X-\mu|>\varepsilon\big) \;\le\; \frac{\sigma^2}{\varepsilon^2}, \tag{21}$$

with no assumption on the shape of the distribution of X.
If $Z_1,\dots,Z_n$ are i.i.d. each with mean μ and variance σ, then

$$\mathrm{Var}(\overline{Z}) \;=\; \frac{1}{n^2}\sum_{i=1}^{n}\mathrm{Var}(Z_i) \;=\; \frac{\sigma^2}{n}. \tag{22}$$

Applying Chebyshev's inequality with $X=\overline{Z}$,

$$P\big(|\overline{Z}-\mu|>\varepsilon\big) \;\le\; \frac{\sigma^2}{n\varepsilon^2}, \tag{23}$$

which recovers consistency with an explicit rate: the probability of a sizeable deviation shrinks as $1/n$, and $\overline{Z}$ concentrates around μ on the scale $\sigma/\sqrt{n}$, its **standard error**.

2.2.2 Specializing to the empirical risk

The empirical risk is an instance of this construction, with g specialized to the loss L and X to the pair $(\mathbf{x},t)$. Let $(\mathbf{x}_1,t_1),\dots,(\mathbf{x}_n,t_n)$ be the pairs in $\mathcal{T}$, drawn i.i.d. from p_J, and let L be a fixed loss. Given a predictor h, let us consider as usual the expected and the empirical risks

$$\mathcal{R}(h)=\mathop{\mathbb{E}}_{(\mathbf{x},t)\sim p_J}\left[L(h(\mathbf{x}),t)\right], \qquad \overline{\mathcal{R}}_{\mathcal{T}}(h)=\frac{1}{n}\sum_{i=1}^{n}L(h(\mathbf{x}_i),t_i). \tag{24}$$

The losses $L(h(\mathbf{x}_i),t_i)$ are i.i.d., and $\overline{\mathcal{R}}_{\mathcal{T}}(h)$ is their sample mean, with $\mathcal{R}(h)$ in the role of μ. Three consequences follow.

Unbiasedness. For any loss and any p_J for which $\mathcal{R}(h)$ exists,

$$\mathop{\mathbb{E}}_{\mathcal{T}\sim p_J^n}\left[\overline{\mathcal{R}}_{\mathcal{T}}(h)\right]=\mathcal{R}(h), \tag{25}$$

the expectation being taken over the random draw of the training set, whatever its size.

Consistency. By the Weak Law of Large Numbers, assuming only that $\mathcal{R}(h)$ exists,

$$\overline{\mathcal{R}}_{\mathcal{T}}(h)\xrightarrow{p}\mathcal{R}(h), \qquad \text{as } |\mathcal{T}|\to\infty. \tag{26}$$

Rate. Writing $\sigma_L^2(h)=\mathop{Var}_{(\mathbf{x},t)\sim p_J}\left[L(h(\mathbf{x}),t)\right]$ for the variance of the per-example loss, we have $\mathop{Var}_{\mathcal{T}\sim p_J^n}\left[\overline{\mathcal{R}}_{\mathcal{T}}(h)\right]=\dfrac{\sigma_L^2(h)}{n}$ and hence

$$P\big(|\overline{\mathcal{R}}_{\mathcal{T}}(h)-\mathcal{R}(h)|>\varepsilon\big) \;\le\; \frac{\sigma_L^2(h)}{n\varepsilon^2}\,: \tag{27}$$

the empirical risk of a *fixed* h concentrates around its expected risk on the scale $\dfrac{\sigma_L(h)}{\sqrt{n}}$.

Our ultimate goal, however, is performance on unseen data, expressed by the expected risk: the relation between the two is the main topic of **statistical learning theory**.

2.3 Minimization of the empirical risk and its issues

The empirical risk allows us to identify the "best" predictor in a given set $\mathcal{H}$ (the **hypothesis class**) via **empirical risk minimization** (ERM),

$$h_{\mathcal{T}} \;=\; \mathop{\mathrm{argmin}}_{h\in\mathcal{H}}\,\overline{\mathcal{R}}_{\mathcal{T}}(h). \tag{28}$$

Two separate questions arise. The first is *statistical*: the predictor we actually want is $h^*_{\mathcal{H}}=\mathop{\mathrm{argmin}}_{h\in\mathcal{H}}\mathcal{R}(h)$, the one with minimal true risk, and we ask whether $h_{\mathcal{T}}$ approaches it as the training set grows. The second is *computational*: $\mathcal{H}$ may be a space of functions in which optimization is hard. We take up the statistical question first, since its answer already tells us which hypothesis classes are admissible.

2.3.1 Reaching the target from a sample

The predictor we would like to find is $h^*_{\mathcal{H}}$, the minimizer of the true risk over $\mathcal{H}$, while ERM returns $h_{\mathcal{T}}$, the minimizer of the empirical risk. Equation (27) ensures that the empirical risk concentrates around the true risk, but only for a predictor h fixed before $\mathcal{T}$ is drawn:

$$P\big(\big|\overline{\mathcal{R}}_{\mathcal{T}}(h) - \mathcal{R}(h)\big| > \varepsilon\big) \xrightarrow{n\to\infty} 0 \qquad \text{for every fixed } h \text{ and } \varepsilon > 0. \tag{29}$$

This guarantee does not cover $h_{\mathcal{T}}$, which is selected by looking at $\mathcal{T}$: a different training set returns, in general, a different predictor. Informally, deriving $h_{\mathcal{T}} \to h^*_{\mathcal{H}}$ from Equation (29) would amount to exchanging a limit and a minimization,

$$\lim_{n\to\infty} \operatorname*{argmin}_{h\in\mathcal{H}} \overline{\mathcal{R}}_{\mathcal{T}}(h) \quad \stackrel{?}{=} \quad \operatorname*{argmin}_{h\in\mathcal{H}} \lim_{n\to\infty} \overline{\mathcal{R}}_{\mathcal{T}}(h), \tag{30}$$

where the right-hand side is $h^*_{\mathcal{H}}$ and the left-hand side is what ERM returns on larger and larger samples. The two operations do not commute in general. Figure 8 shows why: the empirical risk is a noisy version of the true risk, and its minimum may lie where the noise happens to be most favourable rather than where the true risk is smallest.

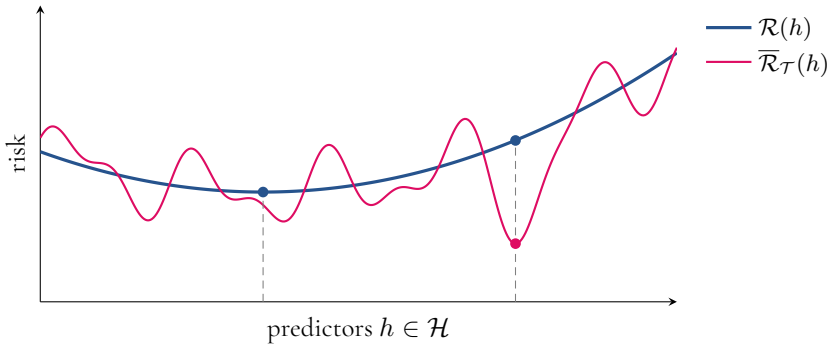


Figure 8: True risk (smooth) and empirical risk (noisy) over a hypothesis class. ERM selects $h_{\mathcal{T}}$, where the empirical risk is lowest: its empirical risk underestimates its true risk, which is larger than that of $h^*_{\mathcal{H}}$.

A sufficient condition for ERM to work follows from comparing the excess risk of $h_{\mathcal{T}}$ with the gaps between empirical and true risk.

Excess risk and uniform deviation

For any training set $\mathcal{T}$, add and subtract the empirical risks of $h_{\mathcal{T}}$ and $h^*_{\mathcal{H}}$:

$$\mathcal{R}(h_{\mathcal{T}}) - \mathcal{R}(h^*_{\mathcal{H}}) = \underbrace{\Big[\mathcal{R}(h_{\mathcal{T}}) - \overline{\mathcal{R}}_{\mathcal{T}}(h_{\mathcal{T}})\Big]}_{\text{(a)}} + \underbrace{\Big[\overline{\mathcal{R}}_{\mathcal{T}}(h_{\mathcal{T}}) - \overline{\mathcal{R}}_{\mathcal{T}}(h^*_{\mathcal{H}})\Big]}_{\text{(b)}} + \underbrace{\Big[\overline{\mathcal{R}}_{\mathcal{T}}(h^*_{\mathcal{H}}) - \mathcal{R}(h^*_{\mathcal{H}})\Big]}_{\text{(c)}}.$$

Term (b) is never positive, since $h_{\mathcal{T}}$ minimizes the empirical risk over $\mathcal{H} \ni h^*_{\mathcal{H}}$, and terms (a) and (c) are bounded in absolute value by $\sup_{h\in\mathcal{H}} |\overline{\mathcal{R}}_{\mathcal{T}}(h) - \mathcal{R}(h)|$. Therefore, for every $\mathcal{T}$,

$$\mathcal{R}(h_{\mathcal{T}}) - \mathcal{R}(h^*_{\mathcal{H}}) \;\le\; 2 \sup_{h\in\mathcal{H}} \big|\overline{\mathcal{R}}_{\mathcal{T}}(h) - \mathcal{R}(h)\big|, \tag{31}$$

and, considering the probability over the draw of $\mathcal{T}$ with $|\mathcal{T}| = n$,

$$P\Big(\mathcal{R}(h_{\mathcal{T}}) - \mathcal{R}(h^*_{\mathcal{H}}) > 2\varepsilon\Big) \;\le\; P\Big(\sup_{h\in\mathcal{H}} \big|\overline{\mathcal{R}}_{\mathcal{T}}(h) - \mathcal{R}(h)\big| > \varepsilon\Big). \tag{32}$$

If the right-hand side of Equation (32) tends to 0 for every $\varepsilon > 0$, that is, if the empirical risk converges to the true risk uniformly over $\mathcal{H}$, the true risk of $h_{\mathcal{T}}$ converges in probability to that of $h^*_{\mathcal{H}}$. The idea is

simple: if no predictor in $\mathcal{H}$ has an empirical risk far from its true risk, then the predictor selected by ERM, whichever it is, has a reliable empirical risk too. Whether uniform convergence holds depends on $\mathcal{H}$.

When $\mathcal{H}$ is unrestricted, the gap between empirical and true risk does not shrink at all. Consider binary classification with $t \in \{0, 1\}$ and the 0–1 loss, let $\mathcal{H}$ contain all functions $\mathcal{X} \to \{0, 1\}$, and let $\mathbf{x}$ have a continuous distribution. Assume the extreme case of noisy labels, $\pi(\mathbf{x}) = p(t = 1 \mid \mathbf{x}) = \frac{1}{2}$ for all $\mathbf{x}$: the label is a fair coin toss, independent of the input. Nothing can then be learned. Any classifier errs with probability $\frac{1}{2}$ on each input, so every $h \in \mathcal{H}$, the Bayes classifier (Section 2.4) included, has true risk $\frac{1}{2}$.

For any training set, however, $\mathcal{H}$ also contains **lookup tables**, which memorize the training set. Consider the simplest one, h_c, which returns t_i on each $\mathbf{x}_i$ and a fixed constant $c \in \{0, 1\}$ everywhere else.

On the training set h_c never errs, since it returns exactly the observed labels: its empirical risk is 0. As the 0–1 risk cannot be negative, this is the smallest possible value, and h_c is an ERM solution. The constant classifier $h(\mathbf{x}) = c$, which does not memorize the data, instead errs on about half of the training points, since each label is a coin toss: its empirical risk is close to its true risk $\frac{1}{2}$. ERM looks only at the empirical risk, and therefore strictly prefers the lookup table: fitting the noise is rewarded.

On new data, however, the two classifiers coincide. Since $\mathbf{x}$ is continuous, a new input coincides with one of the n training points with probability zero. Almost surely, then, h_c predicts c, whatever the input: the memorized labels play no role in prediction, and the true risk of h_c is $\frac{1}{2}$, as for every classifier. ERM thus returns a predictor that looks perfect on the data but does no better than tossing a coin. Nor can more data fix the problem. For every n, h_c has empirical risk 0 and true risk $\frac{1}{2}$, so the largest gap over $\mathcal{H}$ never drops below $\frac{1}{2}$, whatever the sample size: a larger sample only provides a new lookup table with the same property.

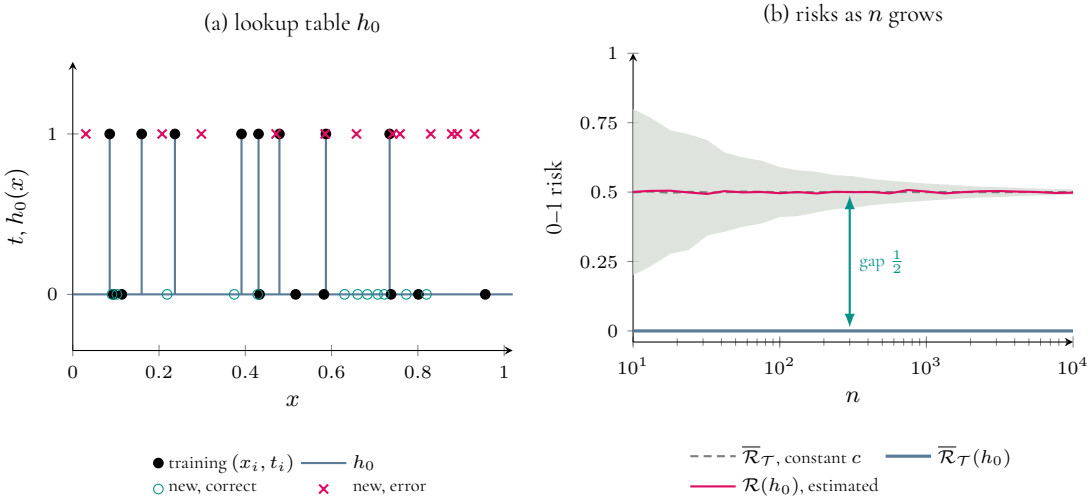


Figure 9: Lookup tables with labels given by fair coin tosses, $x \sim U(0, 1)$, and 0–1 loss. (a) The lookup table h_0 built on $n = 16$ training points: it returns t_i on each x_i, and so never errs on the training set, and 0 everywhere else. New inputs never coincide with training points, so h_0 predicts 0 on all of them, and errs whenever their label is 1: here on 12 of 24 new points. (b) Risks as a function of the training set size n. The empirical risk of h_0 is always 0, while its true risk, estimated on 20000 new points, stays at $\frac{1}{2}$, the risk of every classifier: the gap between the two does not shrink with n. Dashed: empirical risk of the constant classifier $h(x) = 0$, averaged over 4000 training sets, with its central 95% band in grey; it concentrates around its true risk $\frac{1}{2}$.

Lookup tables in detail

Fix a training set $\mathcal{T} = \{(\mathbf{x}_1, t_1), \ldots, (\mathbf{x}_n, t_n)\}$ and $c \in \{0, 1\}$, and define

$$h_c(\mathbf{x}) = \begin{cases} t_i & \text{if } \mathbf{x} = \mathbf{x}_i, \\ c & \text{otherwise,} \end{cases}$$

a function $\mathcal{X} \to \{0, 1\}$, hence in $\mathcal{H}$.

True risk. Since $p(t \neq h(\mathbf{x}) \mid \mathbf{x}) = \frac{1}{2}$ whatever $h(\mathbf{x})$ is, every $h \in \mathcal{H}$ has $\mathcal{R}(h) = \frac{1}{2}$. In particular, a new $\mathbf{x}$ coincides with a training point with probability zero, so almost surely $h_c(\mathbf{x}) = c$, and $\mathcal{R}(h_c) = p(t \neq c) = \frac{1}{2}$.

Empirical risk. On each training point h_c returns exactly t_i, so $\overline{\mathcal{R}}_{\mathcal{T}}(h_c) = 0$, the minimum of the 0–1 risk. For the constant classifier $h(\mathbf{x}) = c$, instead, $\overline{\mathcal{R}}_{\mathcal{T}}(h) = \frac{1}{n}\sum_i \mathbf{1}[t_i \neq c]$ is the average of n independent Bernoulli variables with parameter $\frac{1}{2}$: its mean is $\frac{1}{2}$ and its standard deviation $\frac{1}{2\sqrt{n}}$. With $n = 100$, it lies between 0.4 and 0.6 with probability about 95%.

Consequences. Unlike the quadratic loss, the 0–1 loss is bounded, and so is the gap; it does not decrease, however, since for every $\mathcal{T}$ and every n

$$\sup_{h\in\mathcal{H}} \left|\overline{\mathcal{R}}_{\mathcal{T}}(h) - \mathcal{R}(h)\right| \;\geq\; \left|\overline{\mathcal{R}}_{\mathcal{T}}(h_c) - \mathcal{R}(h_c)\right| \;=\; \tfrac{1}{2},$$

and the right-hand side of Equation (32) never tends to 0.

Restricting $\mathcal{H}$ is therefore necessary for ERM to work. The number of elements is not the right measure of richness, since many infinite classes, such as linear classifiers, converge uniformly: what matters is a suitable complexity measure, such as the VC dimension. This is the first, statistical reason for restricting $\mathcal{H}$; the next subsection gives a second, computational one.

2.3.2 Restricting $\mathcal{H}$ for computational reasons too

A second, independent reason points in the same direction: searching a space of functions for the empirical risk minimizer is not directly amenable to standard optimization tools (gradients, line searches), which are built for spaces of numbers. The standard remedy is to restrict $\mathcal{H}$ to a **parametric family**: fix a functional form $h_{\boldsymbol{\theta}}$ and let $\boldsymbol{\theta} = (\theta_0, \theta_1, \ldots, \theta_d)$ range over $\mathbb{R}^{d+1}$. Each $\boldsymbol{\theta}$ picks out one function in $\mathcal{H}$, the search over functions becomes a search over parameter vectors, and training becomes finding the $\boldsymbol{\theta}$ minimizing the empirical risk — whether $h_{\boldsymbol{\theta}}$ is a linear model, a fixed-depth tree, or a neural network. This restriction is also exactly the kind of complexity control required above: a family indexed by $d+1$ parameters typically has complexity growing only with d, so a modest d makes both gradient-based search and uniform convergence possible.

For the parametric case that occupies most of these notes, we reformulate empirical risk minimization as minimizing the overall loss

$$\mathcal{L}(\boldsymbol{\theta}; \mathcal{T}) = \sum_{i=1}^{n} L_i(\boldsymbol{\theta}), \tag{33}$$

where we sum the individual loss terms $L_i = L(\boldsymbol{\theta}; \mathbf{x}_i, t_i)$ over each data point in the training set, making explicit the dependence on both the parameters and the training example.

2.3.3 Comparing predictors on an independent test set

Every failure traced above has one root cause: the empirical risk was asked to serve *two* roles at once — selecting $h_{\mathcal{T}}$ and certifying its quality — on the same sample. The machinery developed so far (union bound, optimism of the minimum, uniform convergence) repairs this *theoretically*. There is a far cheaper, purely empirical fix: evaluating $h_{\mathcal{T}}$ on a second, independent **test set** $\mathcal{T}'$, drawn i.i.d. from p_J and never consulted during selection. Conditionally on $\mathcal{T}$, $h_{\mathcal{T}}$ is simply *some fixed function* with respect to $\mathcal{T}'$, and the hypothesis of Equation (29) is restored by construction:

$$P\left(\left|\overline{\mathcal{R}}_{\mathcal{T}'}(h_{\mathcal{T}}) - \mathcal{R}(h_{\mathcal{T}})\right| > \varepsilon \;\middle|\; \mathcal{T}\right) \xrightarrow{|\mathcal{T}'|\to\infty} 0, \tag{34}$$

with no supremum over $\mathcal{H}$ and no complexity measure involved. Consequently $\mathcal{H}$ can be as rich as convenient during training — even the interpolating construction above — since $\overline{\mathcal{R}}_{\mathcal{T}'}(h_{\mathcal{T}})$ reports the truth regardless. The gap $\overline{\mathcal{R}}_{\mathcal{T}'}(h_{\mathcal{T}}) - \overline{\mathcal{R}}_{\mathcal{T}}(h_{\mathcal{T}})$ is the practical substitute for the uniform-convergence quantity of Equation (32): generalization is measured after the fact rather than proved in advance.

The fix, however, holds only for a *single*, already-fixed $h_{\mathcal{T}}$ evaluated *once*. If $\mathcal{T}'$ is used to *choose* among K candidates (different algorithms, architectures, or hyperparameters), this is ERM over those candidates with $\mathcal{T}'$ in the role of $\mathcal{T}$, and every result above transfers verbatim: comparing a small K, fixed in advance, once, is the safe regime, whereas repeatedly inspecting performance on $\mathcal{T}'$ and adjusting the model reproduces the optimism of ERM described above. A test set used for model selection has become a second *training* set. Hence the standard three-way split: a *training* set, on which $\mathcal{H}$ can be as rich as convenient; a *validation* set for model and hyperparameter selection, safe only as long as the number of compared configurations stays small;

and a *test* set used exactly once, on the final model — the one point where Equation (29) applies without qualification, giving an unbiased estimate of the true risk.

2.4 Bayes predictors and Bayes risk

The expected risk cannot be minimized in practice, since it requires knowing p_M and p_C. It is nonetheless worth asking what the best possible predictor would look like if this knowledge *were* available: although not computable, it provides an ideal benchmark for learned predictors.

The **Bayes predictor** h^B is defined as the one that at any $\mathbf{x}$ minimizes the risk $\mathcal{R}(\mathbf{x}; h)$ over the class of *all* measurable functions[2] $h : \mathcal{X} \to \mathbb{R}$: Formally,

$$h^B(\mathbf{x}) = \operatorname*{argmin}_h \mathcal{R}(\mathbf{x}; h). \tag{35}$$

As a consequence, the **Bayes predictor** h^B minimizes the expected risk over all measurable functions:

$$h^B = \operatorname*{argmin}_h \mathcal{R}(h). \tag{36}$$

The minimal value $\mathcal{R}^B = \mathcal{R}(h^B)$ is the **Bayes risk**: the error left even for the best conceivable predictor, arising whenever $p_C(\cdot|\mathbf{x})$ spreads mass over more than one value of t. If two patients with identical features sometimes recover and sometimes do not, no predictor can foresee the individual outcome, only report the correct probability of recovery. The Bayes risk is *irreducible*, reflecting noise intrinsic to the phenomenon rather than ignorance that more data could dispel.

Two remarks are in order. First, h^B depends on the loss L, not only on p_J: different losses extract different statistics from the same posterior — squared loss yields the conditional mean, absolute loss the conditional median, and so on.

Example

Suppose, for fixed $\mathbf{x}$, that $t = 0$ with probability 0.9 and $t = 10$ with probability 0.1. Under squared loss the Bayes-optimal prediction is the conditional mean, $0.9 \cdot 0 + 0.1 \cdot 10 = 1$, a value t never takes; under absolute loss it is the conditional median, 0. Same posterior, different losses, different optimal predictions.

Second, computing h^B requires $p_C(\cdot|\mathbf{x})$, exactly what learning assumes to be unavailable: the Bayes predictor is a benchmark, never a practical output of training.

Restricting the search to $\mathcal{H}$ generally worsens the achievable risk, since $\mathcal{H}$ may not contain h^B: no linear function reproduces a genuinely nonlinear relation. Let $h^*_{\mathcal{H}} = \operatorname*{argmin}_{h \in \mathcal{H}} \mathcal{R}(h)$ be the best predictor in $\mathcal{H}$, still computed under perfect knowledge of p_J, and $h_{\mathcal{T}} \in \mathcal{H}$ the predictor actually returned by learning on the finite sample $\mathcal{T}$: the only one of the three we can compute, and generally the worst. Inserting $h^*_{\mathcal{H}}$ as an intermediate reference point decomposes the excess risk of $h_{\mathcal{T}}$ over the Bayes risk:

$$\mathcal{R}(h_{\mathcal{T}}) - \mathcal{R}^B = \underbrace{\left[\mathcal{R}(h_{\mathcal{T}}) - \mathcal{R}(h^*_{\mathcal{H}})\right]}_{\text{estimation error}} + \underbrace{\left[\mathcal{R}(h^*_{\mathcal{H}}) - \mathcal{R}^B\right]}_{\text{approximation error}}. \tag{37}$$

Both terms are nonnegative, since $h^*_{\mathcal{H}}$ and h^B are risk minimizers over their respective search spaces.

The **approximation error**, $\mathcal{R}(h^*_{\mathcal{H}}) - \mathcal{R}^B$, measures the risk given up by searching within $\mathcal{H}$ rather than over all functions. It compares two idealized predictors, so it depends only on how well $\mathcal{H}$ matches the true relation between $\mathbf{x}$ and t, and decreases as $\mathcal{H}$ becomes richer. It is associated with **underfitting**: a restrictive class fails to represent the true relation however carefully it is fit. The **estimation error**, $\mathcal{R}(h_{\mathcal{T}}) - \mathcal{R}(h^*_{\mathcal{H}})$, compares two predictors in the same $\mathcal{H}$ differing only in the information used to select them: exact knowledge of p_J versus a finite, noisy sample. It decreases with more data but increases with model flexibility, since richer classes are harder to estimate reliably; it is the term associated with **overfitting**.

With a smooth quadratic relation plus noise, for instance, linear functions have large approximation error but, having few parameters, small estimation error; degree-ten polynomials have negligible approximation

[2] A function $h : \mathcal{X} \to \mathbb{R}$ is **measurable** if, for every interval $(a, b) \subset \mathbb{R}$, the set $\{\mathbf{x} \in \mathcal{X} : h(\mathbf{x}) \in (a, b)\}$ belongs to the collection of subsets over which probabilities are defined. This is a very mild regularity condition, imposing no constraint on the shape or complexity of h: every function one would realistically compute is measurable, so "all measurable functions" essentially means "all functions".

error but, with few examples, tend to wildly oscillating fits, i.e. large estimation error. Enlarging $\mathcal{H}$ shrinks the first term while inflaming the second: a tension closely related, though not identical, to the bias-variance tradeoff of Section 3. What remains once both controllable errors vanish is the Bayes risk, which no choice of $\mathcal{H}$, amount of data, or algorithm can remove. Recognizing which of the two controllable terms dominates — underfitting or overfitting — is a central diagnostic question of supervised learning.

3 The bias-variance decomposition

The decomposition of Equation (37) concerns the predictor $h_{\mathcal{T}}$ learned from one specific training set. A complementary view analyses the learning algorithm itself: since $h_{\mathcal{T}}$ is random, its risk at any point $\mathbf{x}$ can be studied *on average over training sets*, separating what the procedure systematically predicts (bias) from its sensitivity to the particular sample (variance).

3.1 The general decomposition

Fix an item $\mathbf{x}$ and consider the predictions $h_{\mathcal{T}}(\mathbf{x})$ as $\mathcal{T}$ varies over training sets of size n. Their "center" is summarized by the main prediction

$$\bar{h}(\mathbf{x}) \;=\; \operatorname*{argmin}_{z} \; \mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n} \big[L\big(h_{\mathcal{T}}(\mathbf{x}),\, z\big) \big] \,, \tag{38}$$

the value which, taken as a target, is on average closest to the predictions under L.

Functional relation. If $t = f(\mathbf{x})$, the target is a deterministic function of $\mathbf{x}$, and the expected loss of a prediction z at $\mathbf{x}$ reduces to $L\big(z, f(\mathbf{x})\big)$. For any loss with $L(z,z) = 0 < L(y,z)$ when $y \neq z$, this is minimized only by $z = f(\mathbf{x})$, so that $h^B(\mathbf{x}) = f(\mathbf{x})$. Adding and subtracting $L(\bar{h}(\mathbf{x}), f(\mathbf{x}))$ inside the expectation of the risk over all training sets, and using its linearity together with the fact that $L(\bar{h}(\mathbf{x}), f(\mathbf{x}))$ does not depend on $\mathcal{T}$, gives

$$\begin{aligned}
\mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n} \big[\mathcal{R}\big(\mathbf{x}; h_{\mathcal{T}}\big) \big] &= \mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n} \big[L\big(h_{\mathcal{T}}(\mathbf{x}), f(\mathbf{x})\big) \big] \\
&= \mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n} \big[L\big(\bar{h}(\mathbf{x}), f(\mathbf{x})\big) + L\big(h_{\mathcal{T}}(\mathbf{x}), f(\mathbf{x})\big) - L\big(\bar{h}(\mathbf{x}), f(\mathbf{x})\big) \big] \\
&= \mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n} \big[L\big(\bar{h}(\mathbf{x}), f(\mathbf{x})\big) \big] + \mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n} \big[L\big(h_{\mathcal{T}}(\mathbf{x}), f(\mathbf{x})\big) - L\big(\bar{h}(\mathbf{x}), f(\mathbf{x})\big) \big] \\
&= \underbrace{L\big(\bar{h}(\mathbf{x}), f(\mathbf{x})\big)}_{B(\mathbf{x})} + \underbrace{\mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n} \big[L\big(h_{\mathcal{T}}(\mathbf{x}), f(\mathbf{x})\big) - L\big(\bar{h}(\mathbf{x}), f(\mathbf{x})\big) \big]}_{V(\mathbf{x})} \,.
\end{aligned} \tag{39}$$

Probabilistic relation: including the irreducible risk. If $t \sim p_C(\cdot|\mathbf{x})$, even h^B incurs a nonzero risk. Adding and subtracting the loss of both $h^B(\mathbf{x})$ and $\bar{h}(\mathbf{x})$ inside the expectation over t yields

$$\begin{aligned}
\mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n} \big[\mathcal{R}\big(\mathbf{x}; h_{\mathcal{T}}\big) \big] &= \mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n} \Big[\mathop{\mathbb{E}}_{t\sim p_C(\cdot|\mathbf{x})} \big[L\big(h_{\mathcal{T}}(\mathbf{x}), t\big) \big] \Big] \\
&= \underbrace{\mathop{\mathbb{E}}_{t\sim p_C(\cdot|\mathbf{x})} \big[L\big(h^B(\mathbf{x}), t\big) \big]}_{C(\mathbf{x})} + \underbrace{\mathop{\mathbb{E}}_{t\sim p_C(\cdot|\mathbf{x})} \big[L\big(\bar{h}(\mathbf{x}), t\big) - L\big(h^B(\mathbf{x}), t\big) \big]}_{B(\mathbf{x})} \\
&\quad + \underbrace{\mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n} \Big[\mathop{\mathbb{E}}_{t\sim p_C(\cdot|\mathbf{x})} \big[L\big(h_{\mathcal{T}}(\mathbf{x}), t\big) - L\big(\bar{h}(\mathbf{x}), t\big) \big] \Big]}_{V(\mathbf{x})} \,.
\end{aligned}$$

The three terms involve three different sources of error. The irreducible risk $C(\mathbf{x})$ depends only on the noise of the target around the Bayes prediction $h^B(\mathbf{x})$, and is incurred by every predictor. The bias $B(\mathbf{x})$ is the additional loss incurred by predicting $\bar{h}(\mathbf{x})$ instead of $h^B(\mathbf{x})$: it measures how far the learning procedure is, on average, from the best possible prediction. The variance $V(\mathbf{x})$ is the additional loss due to the predictions $h_{\mathcal{T}}(\mathbf{x})$ scattering around $\bar{h}(\mathbf{x})$ as the training set changes.

Figure **??** illustrates the decomposition in a simple case, with $t = \sin(2\pi x) + \varepsilon$, Gaussian noise of standard deviation 0.3, least-squares lines learned from $n = 10$ items, and quadratic loss. Lines learned from different

training sets differ considerably from each other (panel (a)), and none of them can follow the sinusoid h^B. At the item $\mathbf{x} = 0.2$ two distributions are then involved (panel (b)): that of the target, centered at $h^B(\mathbf{x})$, and that of the predictions $h_{\mathcal{T}}(\mathbf{x})$ over the training sets, centered at their average $\bar{h}(\mathbf{x})$, which is the main prediction for the quadratic loss. Under this loss, as shown below, the three terms take a simple form: $C(\mathbf{x})$ is the variance of the target, $B(\mathbf{x})$ the squared distance between $\bar{h}(\mathbf{x})$ and $h^B(\mathbf{x})$, and $V(\mathbf{x})$ the variance of the predictions. The irreducible risk (0.090) is the noise variance, and would be incurred even by h^B. The bias (0.116) arises because lines are too rigid: on average they pass well below the peak of the sinusoid at $\mathbf{x}$, a systematic error that more training sets would not remove. The variance (0.085) arises because $n = 10$ items are few: the slope and intercept of the learned line, and hence its prediction at $\mathbf{x}$, depend strongly on the particular sample. The three contributions are here of comparable size, and their sum (0.290) is the expected risk at $\mathbf{x}$.

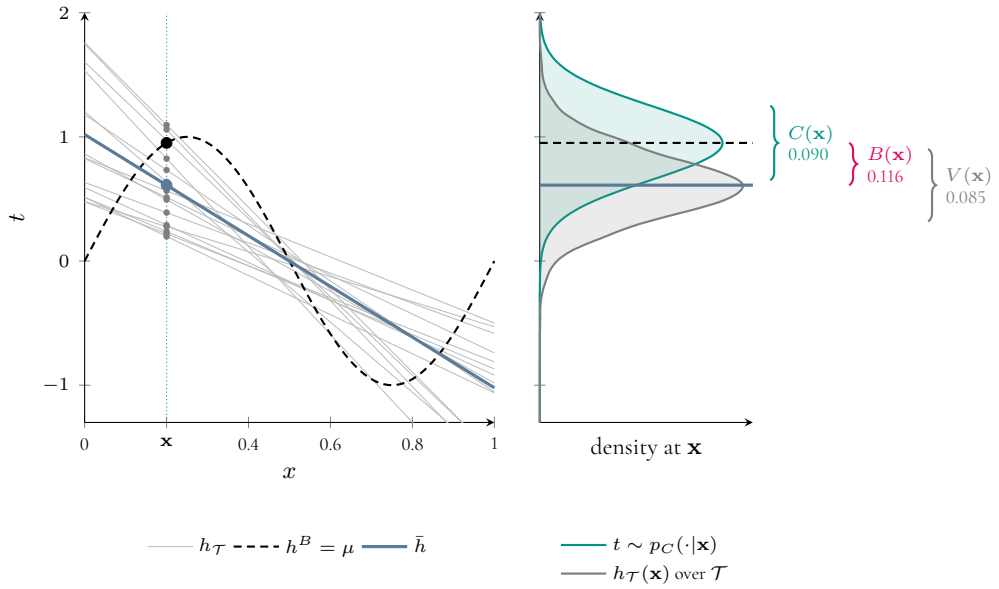


Figure 10: Bias-variance decomposition at a fixed item $\mathbf{x}$, for $t = \sin(2\pi x) + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, 0.3^2)$, least-squares lines learned from $n = 10$ items, and quadratic loss. (a) The Bayes predictor $h^B = \mu$, some predictors $h_{\mathcal{T}}$ learned from different training sets, and the main prediction $\bar{h}$, their average. (b) At $\mathbf{x}$: the distribution of the target $t \sim p_C(\cdot|\mathbf{x})$, centered at $h^B(\mathbf{x})$, and the distribution of the predictions $h_{\mathcal{T}}(\mathbf{x})$ over 5000 training sets, centered at $\bar{h}(\mathbf{x})$. The irreducible risk $C(\mathbf{x})$ is the variance of the target, the bias $B(\mathbf{x})$ the squared distance between $\bar{h}(\mathbf{x})$ and $h^B(\mathbf{x})$, and the variance $V(\mathbf{x})$ the variance of the predictions; the braces show the corresponding standard deviations and distance. Their sum, 0.290, is the expected risk at $\mathbf{x}$.

The identity holds for *every* loss and every choice of $\bar{h}$, since the terms added and subtracted cancel out: by itself, it says nothing about the problem. Its content lies entirely in what the three terms turn out to be. In the example above they have a clear meaning, because the quadratic loss gives them one; as discussed next, this is not the case for every loss.

3.2 Interpretability

$C(\mathbf{x})$ is the irreducible risk , the Bayes risk of Section 2.4, fixed by p_C and L alone. Its value is 0 in the functional, when $h^B(\mathbf{x}) = f(\mathbf{x})$, so it does not appear in Equation (39).

$B(\mathbf{x})$ and $V(\mathbf{x})$ are usually denoted as "bias" and "variance", since they should represent the common behavior of all predictors $h_{\mathcal{T}}$ and how much they tend to differ from the common behavior and between themselves.

For such names to be justified, the terms of the decomposition should satisfy the following properties:

(i) $B(\mathbf{x})$ should depend only on the main prediction $\bar{h}(\mathbf{x})$ sits. Moreover it should be $B(\mathbf{x}) \geq 0$, with $B(\mathbf{x}) = 0$ if and only if $\bar{h}(\mathbf{x}) = h^B(\mathbf{x})$:

(ii) $V(\mathbf{x})$ should depend only on the spread of the predictions $h_{\mathcal{T}}(\mathbf{x})$ around $\bar{h}(\mathbf{x})$, and it shoulb be $V(\mathbf{x}) \geq 0$

(iii) the terms combine additively, so that reducing one, with the others fixed, reduces the total, that is the expectation of the risk.

Property (i) holds for any loss, by definition of h^B. Properties (ii) and (iii) do not: for a general loss the variance term mixes the fluctuation of the predictions with the noise of the target, and can even be negative. There is, however, a broad class of losses for which all three properties hold.

Bregman divergences. Informally, a Bregman divergence measures the cost of predicting y when the truth is t by taking a strictly convex function ϕ and asking how badly its tangent line at y approximates ϕ at t:

$$D_\phi(t, y) \;=\; \phi(t) - \phi(y) - \phi'(y)\,(t - y) \;\geq\; 0. \tag{40}$$

By convexity this gap is never negative, and it vanishes only when $y = t$. Different choices of ϕ give different losses: $\phi(y) = y^2$ gives the quadratic loss $(t - y)^2$.

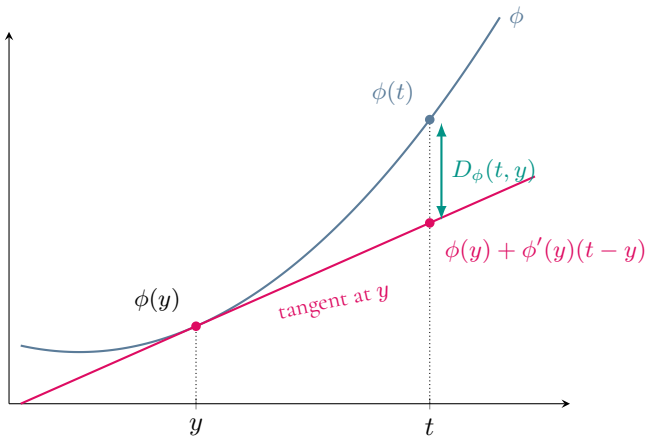


Figure 11: Bregman divergence: the gap at t between the convex function ϕ and its tangent line at y. By convexity the tangent never lies above ϕ, so the gap is non-negative, and it vanishes only when $y = t$.

What makes these losses special is the way their expected loss behaves at a fixed $\mathbf{x}$: for any prediction y, the expected loss splits into a term that depends only on the noise of the target t around the conditional mean $\mu(\mathbf{x}) = \mathop{\mathbb{E}}_{t \sim p_C(\cdot|\mathbf{x})}[t]$, and a term that depends only on how far y is from $\mu(\mathbf{x})$, as measured by D_ϕ:

$$\mathop{\mathbb{E}}_{t \sim p_C(\cdot|\mathbf{x})}\left[D_\phi(t, y)\right] \;=\; \mathop{\mathbb{E}}_{t \sim p_C(\cdot|\mathbf{x})}\left[D_\phi\big(t, \mu(\mathbf{x})\big)\right] \;+\; D_\phi\big(\mu(\mathbf{x}), y\big). \tag{41}$$

In the multivariate case the same steps hold with $\phi'(y)(t-y)$ replaced by the inner product $\nabla\phi(y)^T(t-y)$.

Two consequences follow from (41). The first concerns the Bayes predictor. The first term on the right does not depend on y, and the second is nonnegative and null only for $y = \mu(\mathbf{x})$. The expected loss is therefore minimized by the conditional mean, $h^B(\mathbf{x}) = \mu(\mathbf{x})$, whatever the choice of ϕ. The first term is then the irreducible risk. For $\phi(t) = t^2$, for instance, $D_\phi(t, y) = t^2 - y^2 - 2y(t - y) = (t - y)^2$ is the square loss and (41) becomes the risk decomposition for square loss, that will be discussed later

$$\mathop{\mathbb{E}}_{t \sim p_C(\cdot|\mathbf{x})}\left[(t - y)^2\right] = \mathop{Var}_{t \sim p_C(\cdot|\mathbf{x})}[t] + \big(\mu(\mathbf{x}) - y\big)^2,$$

where the risk is composed by the noise variance plus the squared distance of the prediction from the conditional mean. The second consequence is that the same split can be applied again, this time to the predictions $h_{\mathcal{T}}(\mathbf{x})$ obtained from different training sets. This yields a decomposition of the expected loss into the three components $C(\mathbf{x}), B(\mathbf{x}), V(\mathbf{x})$.

$$\mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n}\left[\mathop{\mathbb{E}}_{t\sim p_C(\cdot|\mathbf{x})}\left[D_\phi\big(t,h_\mathcal{T}(\mathbf{x})\big)\right]\right] = \underbrace{\mathop{\mathbb{E}}_{t\sim p_C(\cdot|\mathbf{x})}\left[D_\phi\big(t,\mu(\mathbf{x})\big)\right]}_{C(\mathbf{x})} + \underbrace{D_\phi\big(\mu(\mathbf{x}),\bar{h}(\mathbf{x})\big)}_{B(\mathbf{x})} + \underbrace{\mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n}\left[D_\phi\big(\bar{h}(\mathbf{x}),h_\mathcal{T}(\mathbf{x})\big)\right]}_{V(\mathbf{x})}.$$

All three terms are non negative, equal to 0 if the two arguments are equal. The bias $B(\mathbf{x})$ compares the main prediction with the Bayes prediction $\mu(\mathbf{x})$, and vanishes only when they coincide. The variance $V(\mathbf{x})$ involves only the predictions, not the target, and vanishes only when all $h_\mathcal{T}(\mathbf{x})$ are equal. Properties (i)–(iii) therefore all hold and the use of thers "bias" and "variance" is justified. In the multivariate case ϕ' is replaced by $\nabla\phi$, and the main prediction is defined by $\nabla\phi(\bar{h}(\mathbf{x})) = \mathop{\mathbb{E}}_{\mathcal{T}}[\nabla\phi(h_\mathcal{T}(\mathbf{x}))]$.

Divergences and the Kullback–Leibler divergence

A **divergence** is a function $D(p,q)$ that measures how much a quantity q differs from a reference quantity p, where p and q may be numbers, vectors, or probability distributions. It is required only that $D(p,q) \geq 0$, with $D(p,q) = 0$ if and only if $p = q$. Unlike a distance, a divergence need not be symmetric, $D(p,q) \neq D(q,p)$ in general, and need not satisfy the triangle inequality. The asymmetry reflects the different roles of the two arguments: p is the reference, typically the truth, and q is the approximation that is being assessed.

The most important divergence between probability distributions is the **Kullback–Leibler (KL) divergence**. For two distributions $p = (p_1,\dots,p_K)$ and $q = (q_1,\dots,q_K)$ over K values, it is defined as

$$KL(p\,\|\,q) = \sum_{k=1}^{K} p_k \log\frac{p_k}{q_k} = \mathop{\mathbb{E}}_{k\sim p}\left[\log\frac{p_k}{q_k}\right],$$

with the sum replaced by an integral for densities. It is the expected log-ratio between the probabilities assigned by p and by q to values drawn from p. In other words, it measures how much less likely, on average, the data generated by p appear when they are evaluated with q instead of p. Its nonnegativity, with equality only for $q = p$, follows from Jensen's inequality applied to the concave logarithm:

$$-KL(p\,\|\,q) = \sum_k p_k \log\frac{q_k}{p_k} \;\leq\; \log\sum_k p_k\frac{q_k}{p_k} = \log\sum_k q_k = 0.$$

Expanding the logarithm of the ratio, the KL divergence splits as

$$KL(p\,\|\,q) = \underbrace{-\sum_k p_k\log q_k}_{\text{cross-entropy } H(p,q)} - \underbrace{\left(-\sum_k p_k\log p_k\right)}_{\text{entropy } H(p)}.$$

The cross-entropy $H(p,q)$ is the expected log loss of the prediction q when the outcomes are distributed according to p. The entropy $H(p)$ does not depend on q. Minimizing the expected log loss with respect to q is therefore equivalent to minimizing $KL(p\,\|\,q)$. The entropy $H(p)$ is the irreducible part, which is attained only by the true distribution $q = p$.

The KL divergence is the Bregman divergence generated by the negative entropy $\phi(p) = \sum_k p_k\log p_k$. Since $\partial\phi/\partial q_k = \log q_k + 1$, we have

$$D_\phi(p,q) = \sum_k p_k\log p_k - \sum_k q_k\log q_k - \sum_k(\log q_k+1)(p_k-q_k) = \sum_k p_k\log\frac{p_k}{q_k} - \sum_k(p_k-q_k),$$

and the last sum vanishes because both p and q sum to one.

Losses with an interpretable decomposition. Several losses widely used in practice are Bregman divergences. Of specific interest here are the **quadratic loss**, $\phi(y) = y^2$, for which $\bar{h}(\mathbf{x})$ is the ordinary mean

of the predictions and the decomposition is the classical noise + bias + variance, and the log loss (cross-entropy) for probabilistic predictors, generated by the negative entropy $\phi(p) = \sum_k p_k \log p_k$, for which, as shown above, D_ϕ is the Kullback–Leibler divergence. Many other common losses — the absolute loss, the Huber loss, the hinge loss, the 0–1 loss — are not Bregman divergences: the decomposition still holds as an identity for them, but its variance term may depend on the noise of the target and may even be negative.

Relation between the two decompositions. The two decompositions look at the same error (the average expected risk over all predictors obtained by ERM over all possible training set of same size n) from different angles. Approximation and estimation errors fix a training set $\mathcal{T}$ (and the predictor $h_\mathcal{T}$ learned from it) and average the pointwise risk of applying $h_\mathcal{T}$ over all possible items, comparing it with the ones of $h^*_\mathcal{H}$ and h^B. Bias and variance fix instead an item $\mathbf{x}$ and average the pointwise risk of predicting at $\mathbf{x}$ over all training sets of the same size. The two can be compared by bringing both to the same quantity, the expected excess risk $\mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n}[\mathcal{R}(h_\mathcal{T})] - \mathcal{R}^B$.

Starting from the bias-variance decomposition, the risk of $h_\mathcal{T}$ is the expectation over $\mathbf{x}$ of its pointwise risk. Since the pair $(\mathbf{x}, t)$ is independent of $\mathcal{T}$, the expectations over $\mathcal{T}$ and over $\mathbf{x}$ can be exchanged, and

$$\begin{aligned}
\mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n}[\mathcal{R}(h_\mathcal{T})] &= \mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n}\left[\mathop{\mathbb{E}}_{\mathbf{x}\sim p_M}\left[\mathop{\mathbb{E}}_{t\sim p_C(\cdot|\mathbf{x})}\left[L\big(h_\mathcal{T}(\mathbf{x}), t\big)\right]\right]\right] \\
&= \mathop{\mathbb{E}}_{\mathbf{x}\sim p_M}\left[\mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n}\left[\mathop{\mathbb{E}}_{t\sim p_C(\cdot|\mathbf{x})}\left[L\big(h_\mathcal{T}(\mathbf{x}), t\big)\right]\right]\right] \\
&= \mathop{\mathbb{E}}_{\mathbf{x}\sim p_M}[C(\mathbf{x}) + B(\mathbf{x}) + V(\mathbf{x})] = \mathcal{R}^B + \mathop{\mathbb{E}}_{\mathbf{x}\sim p_M}[B(\mathbf{x})] + \mathop{\mathbb{E}}_{\mathbf{x}\sim p_M}[V(\mathbf{x})]\,,
\end{aligned}$$

where $\mathop{\mathbb{E}}_{\mathbf{x}\sim p_M}[C(\mathbf{x})] = \mathcal{R}^B$, as the pointwise Bayes risk averages to the Bayes risk. As a consequence, it results:

$$\mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n}[\mathcal{R}(h_\mathcal{T})] - \mathcal{R}^B = \mathop{\mathbb{E}}_{\mathbf{x}\sim p_M}[B(\mathbf{x})] + \mathop{\mathbb{E}}_{\mathbf{x}\sim p_M}[V(\mathbf{x})]\,,$$

Starting instead from Equation (37), the approximation error does not depend on $\mathcal{T}$, and averaging over $\mathcal{T}$ gives

$$\mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n}[\mathcal{R}(h_\mathcal{T})] - \mathcal{R}^B = \mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n}[\mathcal{R}(h_\mathcal{T}) - \mathcal{R}(h^*_\mathcal{H})] + \big(\mathcal{R}(h^*_\mathcal{H}) - \mathcal{R}^B\big).$$

Equating the two expressions of the expected excess risk,

$$\underbrace{\big[\mathcal{R}(h^*_\mathcal{H}) - \mathcal{R}^B\big]}_{\text{approximation error}} + \underbrace{\mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n}[\mathcal{R}(h_\mathcal{T}) - \mathcal{R}(h^*_\mathcal{H})]}_{\text{average estimation error}} = \underbrace{\mathop{\mathbb{E}}_{\mathbf{x}\sim p_M}[B(\mathbf{x})]}_{\text{expected bias}} + \underbrace{\mathop{\mathbb{E}}_{\mathbf{x}\sim p_M}[V(\mathbf{x})]}_{\text{expected variance}}. \tag{42}$$

That is, summing the approximation error and the average (over all training sets) estimation error is the same as summing the expected (over the item population) bias and variance.

Both sides of Equation (42) therefore split the same gap, from $\mathcal{R}^B$ to $\mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n}[\mathcal{R}(h_\mathcal{T})]$, at different intermediate points.

Since[3] $\mathop{\mathbb{E}}_{x\sim p_M}[B(\mathbf{x})] = \mathcal{R}(\bar{h}) - \mathcal{R}^B$ and $\mathop{\mathbb{E}}_{x\sim p_M}[V(\mathbf{x})] = \mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n}\big[\mathcal{R}(h_\mathcal{T}) - \mathcal{R}(\bar{h})\big]$,

$$\begin{aligned}
\mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n}[\mathcal{R}(h_\mathcal{T})] - \mathcal{R}^B &= \big(\mathcal{R}(h^*_\mathcal{H}) - \mathcal{R}^B\big) + \mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n}[\mathcal{R}(h_\mathcal{T}) - \mathcal{R}(h^*_\mathcal{H})] \\
&= \big(\mathcal{R}(\bar{h}) - \mathcal{R}^B\big) + \mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n}\big[\mathcal{R}(h_\mathcal{T}) - \mathcal{R}(\bar{h})\big]\,.
\end{aligned}$$

[3] This derives from:

$$\begin{aligned}
\mathop{\mathbb{E}}_{x\sim p_M}[B(\mathbf{x})] &= \mathop{\mathbb{E}}_{x\sim p_M}\left[\mathop{\mathbb{E}}_{t\sim p_C(\cdot|\mathbf{x})}\left[L\big(\bar{h}(\mathbf{x}), t\big) - L\big(h^B(\mathbf{x}), t\big)\right]\right] \\
&= \mathop{\mathbb{E}}_{x\sim p_M}\left[\mathop{\mathbb{E}}_{t\sim p_C(\cdot|\mathbf{x})}\left[L\big(\bar{h}(\mathbf{x}), t\big)\right]\right] - \mathop{\mathbb{E}}_{x\sim p_M}\left[\mathop{\mathbb{E}}_{t\sim p_C(\cdot|\mathbf{x})}\left[L\big(h^B(\mathbf{x}), t\big)\right]\right] \\
&= \mathcal{R}(\bar{h}) - \mathcal{R}^B
\end{aligned}$$

The approximation/estimation decomposition splits the excess risk at the risk of the best predictor in the class, $h^*_{\mathcal{H}}$, while the bias-variance decomposition splits it at the risk of the main prediction $\bar{h}$.

The terms coincide only when $\bar{h}$ and $h^*_{\mathcal{H}}$ have the same risk, which in general is not the case. The bias compares $\bar{h}$ with h^B, the approximation error compares $h^*_{\mathcal{H}}$ with h^B, and $\bar{h}$ need neither coincide with $h^*_{\mathcal{H}}$ nor even belong to $\mathcal{H}$: an average of neural networks is not, in general, a neural network of the same architecture. The sign of the difference between $\mathcal{R}(h^*_{\mathcal{H}})$ and $\mathcal{R}(\bar{h})$ depends on where $\bar{h}$ lies. If $\bar{h} \in \mathcal{H}$ (as for example happens under the quadratic loss for models linear in their parameters, such as in least squares linear regression) then $\mathcal{R}(\bar{h}) \geq \mathcal{R}(h^*_{\mathcal{H}})$ by definition of $h^*_{\mathcal{H}}$. The expected bias is then at least as large as the approximation error, and the expected variance at most as large as the average estimation error. If instead $\bar{h} \notin \mathcal{H}$, averaging may produce a predictor better than any in the class, and the bias may be smaller than the approximation error. This is the effect exploited by ensemble methods, which approximate $\bar{h}$ by averaging predictors trained on different samples.

An example. The relation between the two decompositions can be observed in a simple simulated problem. Items are drawn as $x \sim U(0,1)$ and targets as $t = \sin(2\pi x) + \varepsilon$, with $\varepsilon \sim \mathcal{N}(0, 0.3^2)$, and the quadratic loss is used throughout. The conditional mean is $\mu(x) = \sin(2\pi x)$, which is therefore the Bayes predictor h^B, and the Bayes risk is the noise variance, $\mathcal{R}^B = 0.09$. Since the quadratic loss is a Bregman divergence, the risk of any predictor h is $\mathcal{R}(h) = 0.09 + \mathbb{E}_x\left[(h(x) - \mu(x))^2\right]$, and can be computed by numerical integration over x. Two hypothesis classes are compared, both learned by ERM with the quadratic loss, that is by least squares:

- **lines**, $h(x) = w_0 + w_1 x$, learned from training sets of size $n = 10$; the ERM solution is the ordinary least-squares fit;
- **regression stumps**, the simplest regression trees, which split the input range at a single threshold s and predict a constant on each side: $h(x) = c_1$ for $x < s$ and $h(x) = c_2$ for $x \geq s$. For a fixed s, the empirical risk is minimized by taking c_1 and c_2 equal to the average target of the training items on the two sides; s is then chosen, among the midpoints between consecutive training items, as the one with the smallest training squared error. Training sets have size $n = 20$.

In both cases $h^*_{\mathcal{H}}$ is obtained by minimizing the true risk over the class. The expectations over $\mathcal{T}$ are estimated over 5000 training sets.

Under the quadratic loss the main prediction is the mean of the predictions,

$$\bar{h}(x) = \mathop{\mathbb{E}}_{\mathcal{T}}\left[h_{\mathcal{T}}(x)\right] :$$

at each x, $\bar{h}$ predicts the average of the values predicted by the learned predictors $h_{\mathcal{T}}$, and is estimated by averaging the 5000 predictions pointwise. Figure 11 shows how differently this average behaves in the two classes. For lines, the prediction is linear in the parameters, so that $\mathbb{E}_{\mathcal{T}}\left[w_{0,\mathcal{T}} + w_{1,\mathcal{T}} x\right] = \mathbb{E}_{\mathcal{T}}[w_{0,\mathcal{T}}] + \mathbb{E}_{\mathcal{T}}\left[w_{1,\mathcal{T}}\right] x$: the average prediction is the prediction of the line with averaged parameters, and $\bar{h} \in \mathcal{H}$. It does not, however, coincide exactly with $h^*_{\mathcal{H}}$: each $h_{\mathcal{T}}$ is the line that best fits μ at the few sampled items only, and since μ is not a line, this best fit depends on where the items happen to fall. Averaging lines fitted to small samples is not the same as fitting a single line to the whole population, and the two results differ slightly; they coincide when μ is itself linear, and get closer as n grows. Here the difference is very small, and the two lines are almost indistinguishable.For stumps, $\bar{h}(x)$ averages, at each x, the left constant $c_{1,\mathcal{T}}$ of the stumps whose threshold lies beyond x and the right constant $c_{2,\mathcal{T}}$ of the others. Since the thresholds $s_{\mathcal{T}}$ vary with the training set, the average of the steps changes gradually from the left to the right level, with a profile reflecting the distribution of $s_{\mathcal{T}}$ around $1/2$: $\bar{h}$ is a smooth function, not a stump, and follows μ more closely than the best stump.

$$\begin{aligned}
\mathop{\mathbb{E}}_{x\sim p_M}[V(\mathbf{x})] &= \mathop{\mathbb{E}}_{x\sim p_M}\left[\mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n}\left[\mathop{\mathbb{E}}_{t\sim p_C(\cdot|\mathbf{x})}\left[L\big(h_{\mathcal{T}}(\mathbf{x}), t\big) - L\big(\bar{h}(\mathbf{x}), t\big)\right]\right]\right] \\
&= \mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n}\left[\mathop{\mathbb{E}}_{x\sim p_M}\left[\mathop{\mathbb{E}}_{t\sim p_C(\cdot|\mathbf{x})}\left[L\big(h_{\mathcal{T}}(\mathbf{x}), t\big) - L\big(\bar{h}(\mathbf{x}), t\big)\right]\right]\right] \\
&= \mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n}\left[\mathop{\mathbb{E}}_{x\sim p_M}\left[\mathop{\mathbb{E}}_{t\sim p_C(\cdot|\mathbf{x})}\left[L\big(h_{\mathcal{T}}(\mathbf{x}), t\big)\right]\right] - \mathop{\mathbb{E}}_{x\sim p_M}\left[\mathop{\mathbb{E}}_{t\sim p_C(\cdot|\mathbf{x})}\left[L\big(\bar{h}(\mathbf{x}), t\big)\right]\right]\right] \\
&= \mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n}\left[\mathcal{R}(h_{\mathcal{T}}) - \mathcal{R}(\bar{h})\right]
\end{aligned}$$

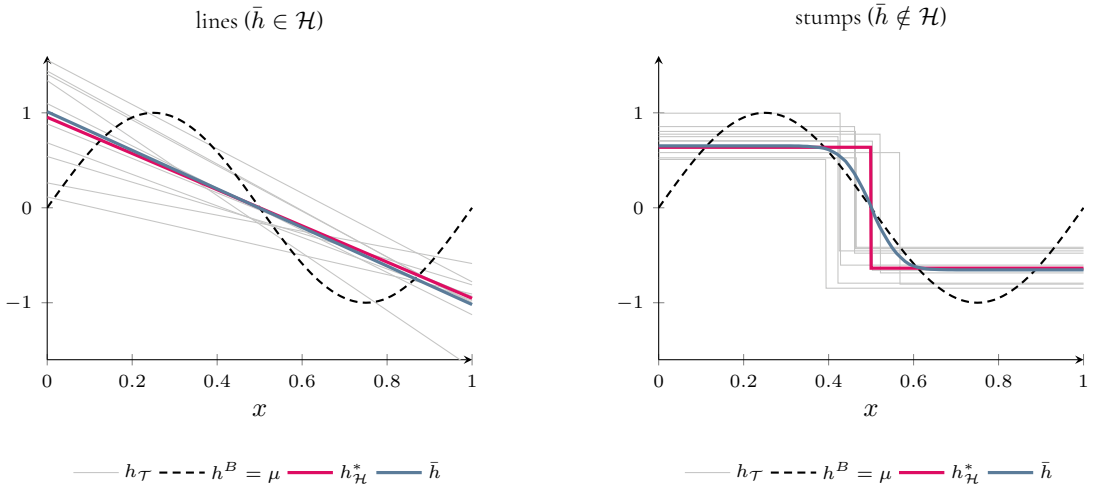


Figure 12: Main prediction $\bar{h}$ and best predictor in the class $h^*_{\mathcal{H}}$, for $t = \sin(2\pi x) + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, 0.3^2)$, and quadratic loss. Grey: ten predictors $h_{\mathcal{T}}$ learned by least squares from different training sets; $\bar{h}$ is their pointwise average over 5000 training sets. Left: lines, $n = 10$; the average of lines is a line, close to $h^*_{\mathcal{H}}$. Right: regression stumps, $n = 20$; the average of steps with varying thresholds is a smooth function, outside the class.

Figure 12 reports, for the two classes, the risks of the four predictors involved and the corresponding terms of the two decompositions.

- For lines, the average risk of the learned predictors is $\mathop{\mathbb{E}}_{\mathcal{T}}[\mathcal{R}(h_{\mathcal{T}})] = 0.367$, so that the expected excess risk is 0.277. It splits into an approximation error of 0.196 and an average estimation error of 0.080, or into an expected bias of 0.198 and an expected variance of 0.079. Since $\bar{h} \in \mathcal{H}$, its risk $\mathcal{R}(\bar{h}) = 0.288$ cannot be smaller than $\mathcal{R}(h^*_{\mathcal{H}}) = 0.286$, and as the two predictors are close the two splits nearly coincide: bias and variance are here essentially approximation and estimation error.
- For stumps, $\mathop{\mathbb{E}}_{\mathcal{T}}[\mathcal{R}(h_{\mathcal{T}})] = 0.222$ and the expected excess risk is 0.132, split into an approximation error of 0.095 and an average estimation error of 0.037, or into an expected bias of 0.067 and an expected variance of 0.065. Here $\mathcal{R}(\bar{h}) = 0.157$ is smaller than $\mathcal{R}(h^*_{\mathcal{H}}) = 0.185$: the average of the stumps is a better predictor than any stump, and the expected bias is smaller than the approximation error by 0.028, the same amount by which the expected variance exceeds the average estimation error. The two decompositions thus disagree on how to classify part of the error of stumps. The approximation/estimation decomposition attributes the whole gap $\mathcal{R}(h^*_{\mathcal{H}}) - \mathcal{R}^B = 0.095$ to the limits of the class: no single stump can follow μ more closely than $h^*_{\mathcal{H}}$. The bias-variance decomposition measures instead the bias from $\bar{h}$, which is not a stump and approximates μ better than any stump, and counts the remaining 0.028 as variance. This is not an arbitrary choice: that part of the error is due to relying on a single stump learned from a single training set, and is removed by averaging stumps learned from different training sets, which brings the risk down to $\mathcal{R}(\bar{h}) = 0.157$, below that of the best stump. What looks like a limit of the class is thus partly removable by averaging, which is the idea underlying ensemble methods such as bagging.

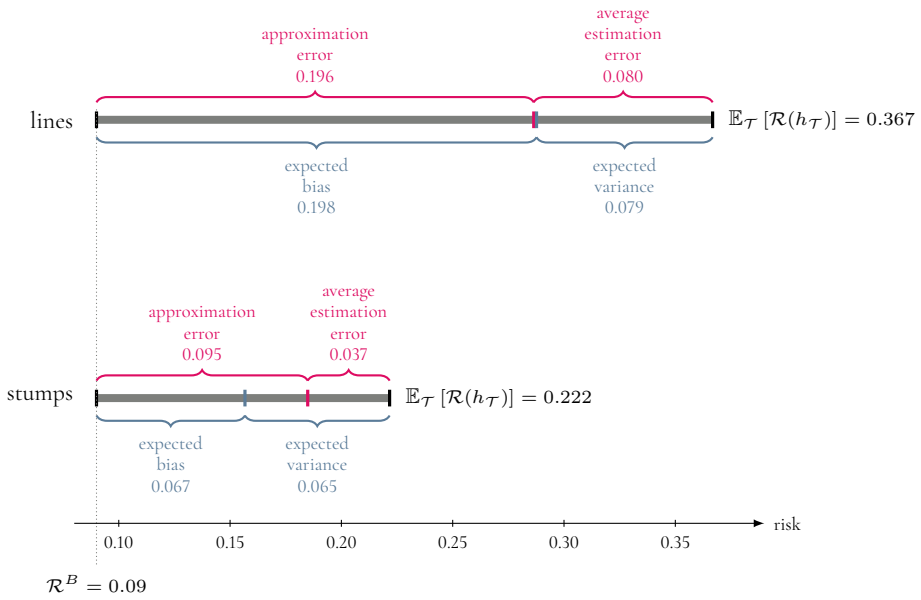


Figure 13: The two decompositions of the expected excess risk, for the lines and stumps of Figure 11, as two splits of the same interval, from $\mathcal{R}^B = 0.09$ to the average risk $\mathbb{E}_{\mathcal{T}}[\mathcal{R}(h_{\mathcal{T}})]$ of the learned predictors. Above each interval (red): approximation error and average estimation error, split at $\mathcal{R}(h^*_{\mathcal{H}})$ (red mark). Below (blue): expected bias and expected variance, split at $\mathcal{R}(\bar{h})$ (blue mark).